Framework côté client — Vue.js 3

TP 1 — Découverte des API REST et consultation des tickets

Durée indicative : 2 heures | Travail individuel | Technologies : Vue 3, Vite, Axios, JSON Server

Contexte et objectifs

Une équipe de support informatique souhaite suivre les incidents déclarés par ses utilisateurs. Chaque ticket comporte : id, titre, description, demandeur, priorité, statut et dateCreation.

À l'issue du TP, l'étudiant saura communiquer avec une API REST simulée et organiser une interface Vue.js réactive.

Rappel : qu'est-ce qu'une API REST ?

Une API REST expose des ressources à travers des URL HTTP. Ici, la ressource « tickets » est accessible via http://localhost:3000/tickets. Le client Vue envoie des requêtes et reçoit des réponses JSON.

- GET /tickets : lire la liste ; GET /tickets/1 : lire un ticket.
- POST /tickets : créer ; PUT /tickets/1 : remplacer ; PATCH /tickets/1 : modifier partiellement ; DELETE /tickets/1 : supprimer.
- Codes courants : 200 (succès), 201 (création), 204 (succès sans contenu), 404 (introuvable), 500 (erreur serveur).

JSON est un format texte structuré en objets et tableaux ; il n'est pas une base de données à lui seul. JSON Server fournit une API locale à partir d'un fichier JSON.

Mise en place de JSON Server

1. Vérifier Node.js et npm ; créer le projet Vue avec : npm create vite@latest tickets-maintenance -- --template vue

2. Entrer dans le dossier : cd tickets-maintenance ; puis npm install ; npm install axios

3. Créer le fichier db.json à la racine du projet avec le contenu ci-dessous :

{ "tickets": [{ "id": "1", "titre": "Imprimante bloquée", "description": "Bourrage papier au secrétariat", "demandeur": "Amira", "priorite": "Haute", "statut": "Ouvert", "dateCreation": "2026-11-01" }, { "id": "2", "titre": "Wi-Fi indisponible", "description": "Connexion instable en salle B", "demandeur": "Sami", "priorite": "Moyenne", "statut": "En cours", "dateCreation": "2026-10-30" }, { "id": "3", "titre": "Écran noir", "description": "PC du laboratoire ne démarre pas", "demandeur": "Leila", "priorite": "Basse", "statut": "Résolu", "dateCreation": "2026-10-29" }] }

4. Démarrer l'API dans un terminal : npx json-server db.json --port 3000

5. Ouvrir http://localhost:3000/tickets dans le navigateur et vérifier que les trois tickets sont renvoyés au format JSON.

6. Dans un second terminal, démarrer Vue avec npm run dev. Garder les deux serveurs actifs.

Travail demandé — interface Vue.js

- Créer un composant TicketList.vue qui affiche les tickets (titre, demandeur, priorité, statut, date).
- Créer src/services/ticketService.js ; configurer Axios avec une URL de base et une méthode getTickets().
- Dans le composant, utiliser ref(), onMounted() et async/await pour charger les données.
- Afficher « Chargement… » pendant la requête et un message explicite en cas d'erreur (par exemple serveur arrêté).
- Ajouter un champ de recherche par titre et deux filtres (statut et priorité) avec computed().
- Afficher le nombre de tickets visibles et un message lorsque la liste filtrée est vide.
- Mettre en forme les priorités et statuts par des badges visuels accessibles.

Captures d'écran obligatoires

- Capture 01 : navigateur affichant la réponse JSON de GET /tickets.
- Capture 02 : application Vue affichant les trois tickets.
- Capture 03 : résultat d'une recherche par titre.
- Capture 04 : filtrage combiné statut et priorité.
- Capture 05 : message d'erreur lorsque JSON Server est arrêté.

Questions à traiter dans le README

Quelle différence entre une API REST et le fichier db.json ? Pourquoi utiliser onMounted() ? Quel est le rôle d'Axios ? Quelle différence entre ref() et computed() ?

Modalités de rendu — GitHub

- Créer un dépôt GitHub public nommé tp1-tickets-maintenance-nom-prenom (ou privé avec accès accordé à l'enseignant).
- Pousser le code source, package.json, package-lock.json, db.json et README.md. Ne pas versionner node_modules, dist ni les secrets.
- Le README doit préciser : identité, prérequis, installation (npm install), lancement du serveur JSON (npx json-server db.json --port 3000), lancement de Vue (npm run dev), fonctionnalités réalisées et difficultés rencontrées.
- Ajouter dans le dépôt un dossier captures/ contenant les captures demandées, numérotées et lisibles.
- Déposer le lien GitHub sur la plateforme du cours avant la date communiquée par l'enseignant.

Critères d'évaluation (sur 20)

- Configuration et fonctionnement de l'API simulée : 4 pts
- Affichage des données et état de chargement : 6 pts
- Recherche et filtres : 5 pts
- Qualité du code, README et captures : 5 pts

Consigne : l'application doit être démontrable localement ; les captures seules ne remplacent pas le code fonctionnel.